Bluetooth Controlled Unmanned Ground Vehicle (UGV)

Sushmitha Ankireddy EE26BTECH11229

V Keerthana EE26BTECH11232

Anvidha Reddy EE26BTECH11221

1. Aim

To build a small ground vehicle that is controlled wirelessly from a smartphone over Bluetooth, with its speed and direction controlled using PWM.

2. Components Used

S.No.	Component	Qty
1	Chassis frame	1
2	Wheels	2
3	Caster wheel	1
4	DC motors	2
5	ESP32 development board	1
6	L293D motor driver IC	1
7	Small breadboard	1
8	Jumper wires (M-M and M-F)	As required
9	18650 battery	1
10	Micro-USB cable	1
11	Smartphone with Dabble app	1
12	BEC Buck Converter	1

2.1 DC motors (2)

These convert electrical energy into rotation and make the wheels turn. The direction of rotation depends on the polarity of the voltage applied, and the speed depends on the average voltage supplied. The vehicle

uses one motor on each side. Turning is done by running the two motors in different directions or at different speeds.

2.2 ESP32 development board (1)

This is the main controller of the vehicle. It has Bluetooth built in, so it receives the commands sent from the phone. It then decides what the motors should do and sends control signals to the motor driver. It also generates the PWM signals that control the speed.

2.3 L293D motor driver IC (1)

The ESP32 pins can only supply a very small current, which is not enough to run motors directly. The L293D works as a link between the ESP32 and the motors. It contains two H-bridge circuits, one for each motor.

- The input pins (IN1 to IN4) decide the direction of each motor. Reversing the inputs reverses the motor.
- The enable pins (EN1,2 and EN3,4) receive the PWM signal from the ESP32. A higher duty cycle gives the motor more average voltage, so it runs faster.
- It takes the motor power from a separate supply, so the motors do not load the ESP32.

2.4 Small breadboard (1)

This is used to make the connections between the ESP32, the L293D and the battery without soldering. It also gives a common ground rail, and it makes the circuit easy to change or correct.

2.5 18650 battery (1)

This is the power supply for the motors. It is connected to the motor supply pin (VCC2) of the L293D. Using a separate battery for the motors keeps the ESP32 from resetting when the motors draw high current.

2.6 Micro-USB cable (1)

This is used to upload the code from the computer to the ESP32 and to power the ESP32 while testing.

2.7 Smartphone with Dabble app

The Dabble app is used as the remote control. Its Gamepad module sends button presses to the ESP32 over Bluetooth. The direction buttons give forward, backward, left and right, and two other buttons increase or decrease the speed.

2.8 BEC Buck Converter

We used the BEC buck converter to step down the 7.4V output from the 2 × 18650 battery pack to a stable 5V. This safe, regulated supply powers both the ESP32 microcontroller and the L293D motor driver.

Why it is better than a 7805

- **Saves battery power:** A 7805 wastes extra voltage by turning it into heat, but the BEC buck converter converts the power efficiently so the batteries last much longer.
- **Does not get hot:** Dropping 7.4V to 5V makes a 7805 get very hot and cut off, while the BEC stays cool without needing a heat sink.

Why it is better than a LM2596

- **Clean power for microcontrollers:** Standard buck converters can send electrical noise into the circuit, but a BEC has built-in filters to give clean power to the ESP32.
- **Prevents sudden restarts:** The BEC easily handles sudden power jumps from the motors, keeping the ESP32 running smoothly without restarting.

3. Circuit Diagram

> **System flow:** Smartphone → Bluetooth → ESP32 → L293D → DC Motors → UGV Movement

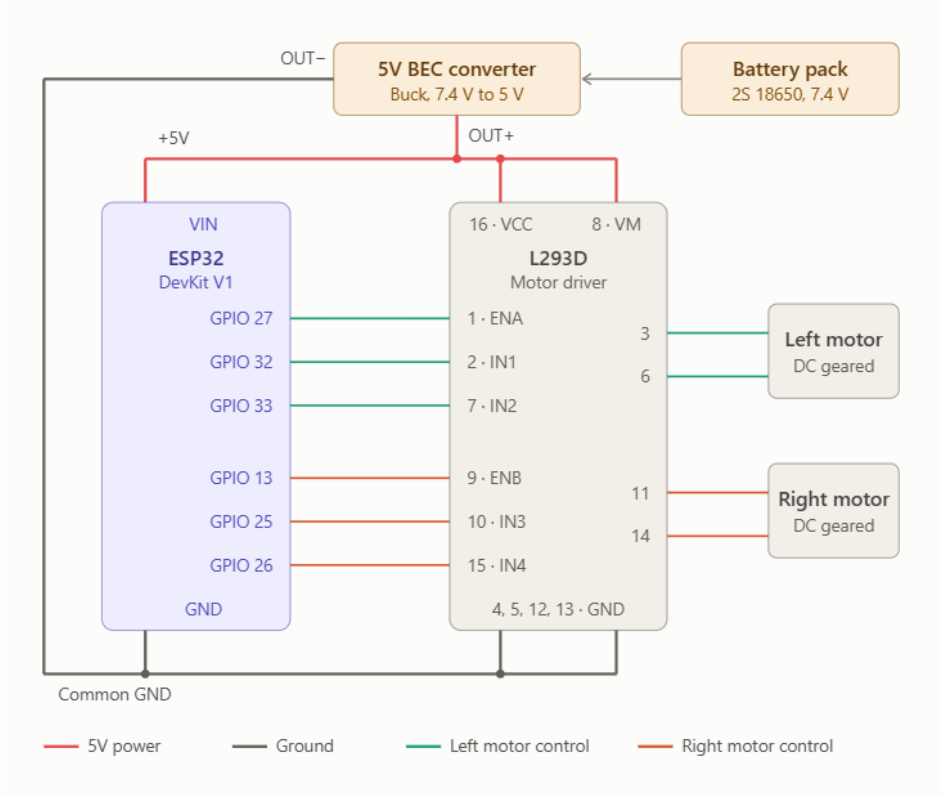


Figure 1: Circuit diagram of the UGV

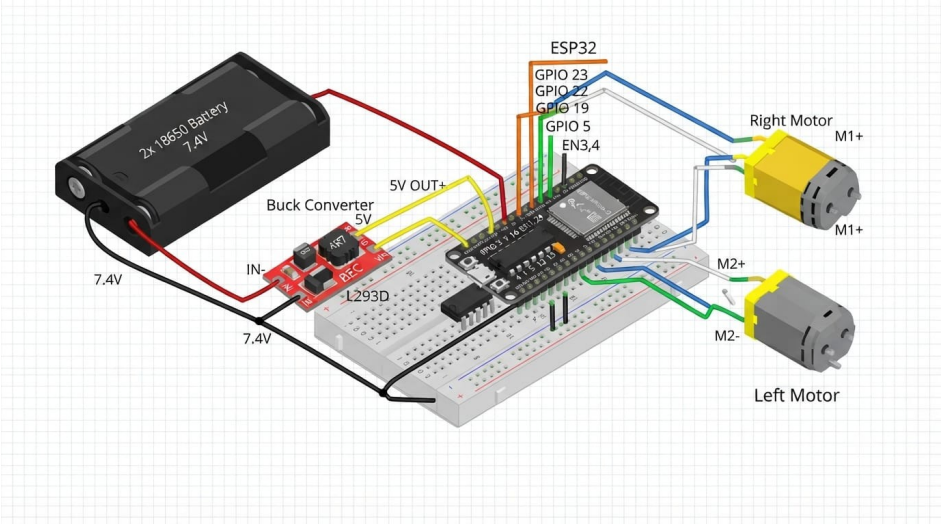


Figure 2: Breadboard wiring view

4. Pin Connections

The enable pins are connected to PWM-capable ESP32 GPIO pins. A common ground is maintained between the ESP32, BEC and L293D.

Table 1: ESP32 and L293D Pin Connections

Function	ESP32 GPIO	L293D Pin
Left Motor Enable (ENA)	27	1
Left Motor Input 1 (IN1)	32	2
Left Motor Input 2 (IN2)	33	7
Right Motor Enable (ENB)	13	9
Right Motor Input 3 (IN3)	25	10
Right Motor Input 4 (IN4)	26	15

Table 2: L293D Connections

L293D Pin	Connection
3 and 6	Left motor terminals 1 and 2
11 and 14	Right motor terminals 1 and 2
8	Motor supply
16	Logic supply
4, 5, 12, 13	Common GND

5. Development Journey

Phase 1: Understanding the hardware and planning. The system was divided into three sections: power supply, processing and communication, and motor control. The planned operation was Smartphone → Bluetooth → ESP32 → L293D → DC Motors → UGV movement.

Phase 2: Initial ESP32 setup and Bluetooth testing. We had difficulty pairing over Bluetooth. The ESP32 supports both Bluetooth Classic and BLE, but the DabbleESP32 library uses BLE, so the Classic pairing procedure did not establish the Dabble connection. We installed the DabbleESP32 library and used the nRF Connect BLE scanner to check that the phone could see the ESP32.

Phase 3: Replacing the ESP32 module. Bluetooth connectivity stayed inconsistent. To isolate the hardware issue, we obtained another ESP32

DevKit V1 and tested it on USB power with a simple BLE advertising program. It appeared in nRF Connect as NEW_ESP32_TEST, confirming that it could advertise over BLE. We continued the Dabble connection with the replacement module.

Phase 4: Power supply design and BEC debugging. Power management was the most challenging part of the project. The 2S pack gives about 7.4 V (8.4 V when full), so a 5V BEC buck converter (IN+, IN−, OUT+, OUT−) supplies the ESP32. At first, the BEC output wires were connected with jumper wires and not soldered, which could cause loose or intermittent connections. One BEC was also accidentally shorted and had to be replaced. This phase showed the importance of proper soldering, avoiding short circuits, keeping a common ground, and checking supply voltage before connecting components.

Phase 5: Motor driver connections. After stabilising the power supply, we connected the two DC motors to the L293D as listed in the pin tables above.

Phase 6: Independent motor testing. A simple test program checked that both motors rotated, ran forward and reverse, and stopped correctly. Later, one wheel occasionally failed to start and needed a manual push. We checked that the wheels were not rubbing against the chassis and experimented with the PWM value. The initial working

value was 160, and the motors started reliably at it, so we kept it for the first movement tests.

Phase 7: Integrating Dabble Bluetooth control. We used the Dabble GamePad through the DabbleESP32 library. Bluetooth disconnected frequently at first. We traced this to loose BEC output wiring, and after improving the connections and stabilizing the power, communication became reliable. The UGV then responded to GamePad commands.

Phase 8: Multiple speed modes. We added three speed modes, selected with GamePad buttons. The PWM value is sent to the enable pins using analogWrite(). A higher PWM value gives a higher average motor voltage and speed, subject to motor load and driver voltage drop.

Phase 9: Smooth acceleration and deceleration. The program now changes speed gradually in steps of 15 per update instead of jumping to the target. For example, from 0 to a target of 200 the speed goes 0 → 15 → 30 → 45 → … → 195 → 200. The same happens in reverse when stopping, and independently for each motor.

Phase 10: Safety mechanism and final integration. A command inactivity timer checks whether directional commands have been received within 500 milliseconds. If not, the target motor speeds are set to zero. The final program combines Bluetooth communication,

direction control, three speed modes, PWM motor control, smooth ramping and the timeout. It was tested successfully with Dabble.

6. Control Logic

Direction Control Logic

GamePad Input	Left Motor	Right Motor
UP	Forward	Forward
DOWN	Reverse	Reverse
LEFT	Reverse	Forward
RIGHT	Forward	Reverse
No directional input	Stop	Stop

Speed Control Modes

Mode	Dabble Button	PWM Value
Slow	SELECT	120
Normal (default)	TRIANGLE	200
Turbo	START	240

7. Challenges Faced and Lessons Learnt

- **Bluetooth communication problems:** pairing and discovery were inconsistent. We learnt the difference between Bluetooth Classic and BLE, and that DabbleESP32 uses BLE.

- **ESP32 module replacement:** testing the replacement module on its own with a BLE scanner helped isolate the communication hardware from the rest of the circuit.
- **BEC short circuit:** one converter was accidentally shorted and replaced. We learnt to check polarity and avoid accidental contact between positive and negative terminals.
- **Loose power connections:** the BEC output was not soldered at first. Improving the connections gave more reliable power delivery.
- **Motor starting problems:** one wheel sometimes failed to start. We checked for mechanical interference and tuned the PWM value.
- **Arduino compilation errors:** duplicate functions and misplaced code caused errors. We fixed this by replacing the sketch with one clean program with a single setup() and loop().

8. Conclusion

The project successfully demonstrates a Bluetooth-controlled Unmanned Ground Vehicle built around an ESP32. Together, the ESP32, L293D motor driver, DC geared motors and Dabble app allow wireless control of the vehicle. PWM gave three selectable speed modes, and smooth ramping improved the transitions between speeds. The project also gave practical experience in hardware assembly, power

management, Bluetooth communication, motor driver interfacing, embedded programming and debugging.

9. We Should Actually Give 7.4V (Batteries) to Pin 8

Internal voltage drop: Because the L293D drops around 1.5V–2V across its internal transistors, feeding 7.4V to Pin 8 means the motors will actually receive roughly 5.5V–6V.

What happens if you feed 5V to Pin 8: If you connect the buck converter's 5V output to Pin 8, the motors will only get 3.0V–3.5V. This will make the car drive very slowly.

Separation of power: Feeding 7.4V from the battery to Pin 8 makes sure the motors get good enough voltage and move faster.

But to do this, we had to put Pin 8 on a breadboard and from there somehow connect it to the buck converter and later to the ESP32. We were not completely sure if that works or not, because when we use that many jumper wires while the current is very low, we didn't know if too much resistance would affect it.

So, to get around that problem, we just modified the code and increased its speed, but we would still like to give the different voltages separately as that is more effective.

10. Future Scope

- **Obstacle detection and avoidance:** An ultrasonic sensor can be added to detect obstacles in the vehicle’s path and help it avoid collisions.
- **Autonomous navigation:** GPS and other navigation sensors can be integrated to enable the UGV to follow predefined paths without continuous human control.
- **Improved wireless control:** The existing Bluetooth-based control system can be extended using Wi-Fi or other communication technologies to increase the operating range.